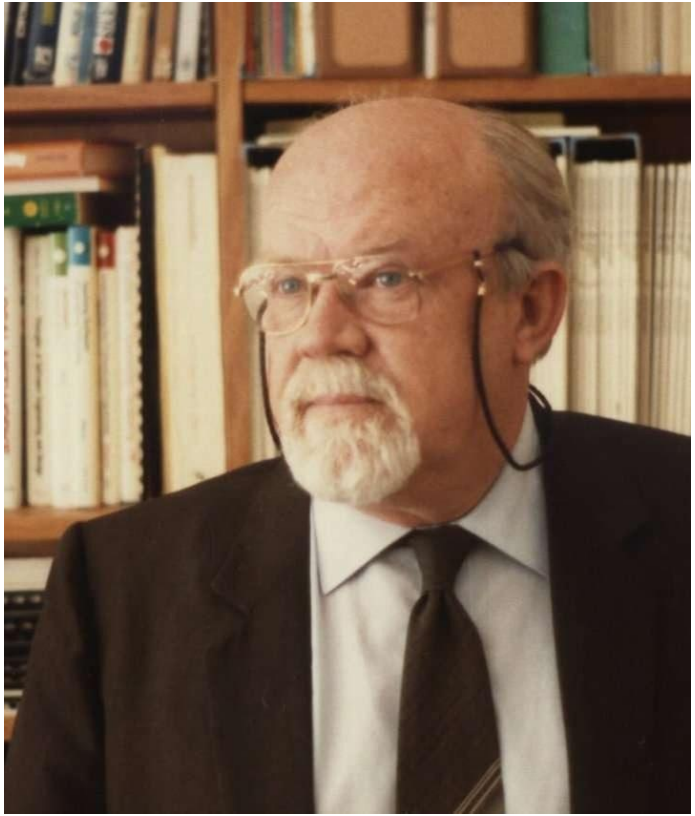
# Use a Rust SAT Solver to play better pickleball!

Bob McNaughton
Rust NYC April 21, 2026
NDC Toronto May 8, 2026



# Why?

- Cook-Levin Theorem
- Useful for proving other results
- Weed out class?

# The Complexity of Theorem-Proving Procedures

Stephen A. Cook

University of Toronto

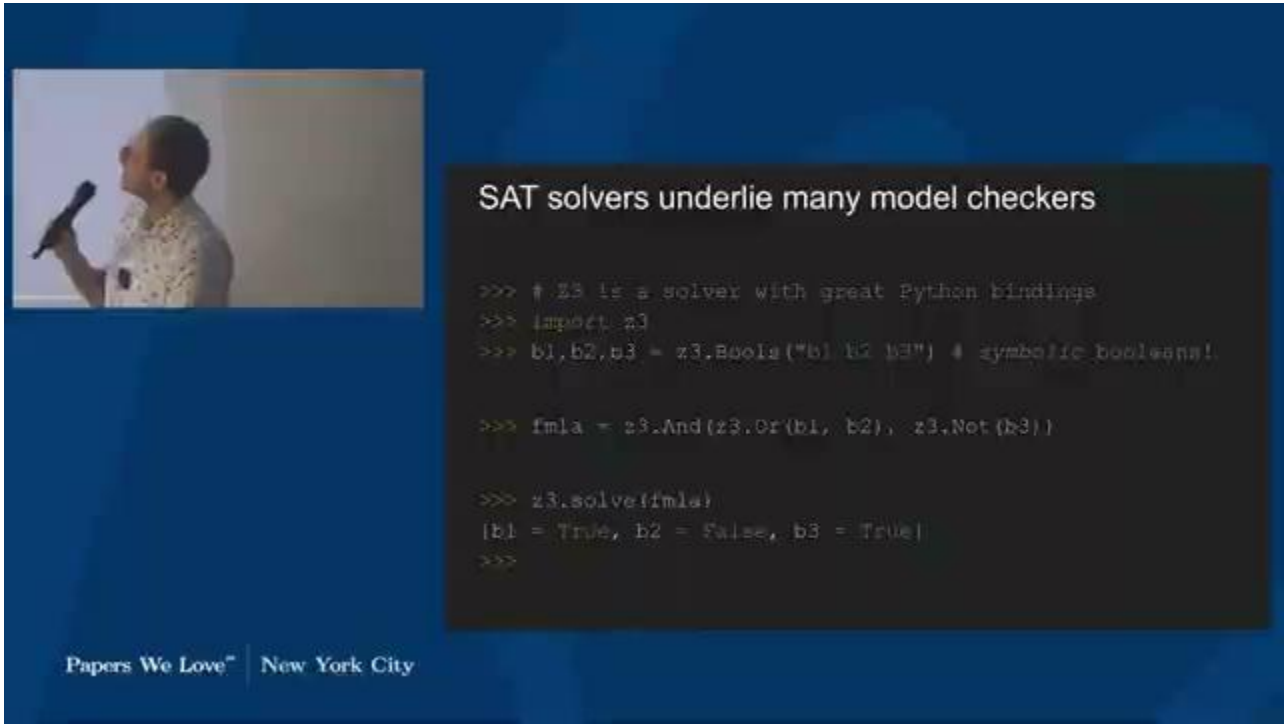
Summary

It is shown that any recognition problem solved by a polynomial time-bounded nondeterministic Turing machine can be "reduced" to the problem of determining whether a given propositional formula is a tautology. Here "reduced" means, roughly speaking, that the first problem can be solved deterministically in polynomial time provided an oracle is available for solving the second. From this notion of reducible, polynomial degrees of difficulty are defined, and it is shown that the problem of determining tautologyhood has the same polynomial degree as the problem of determining whether the first of two given graphs is isomorphic to a subgraph of the second. Other examples are discussed. A method of measuring the complexity of proof procedures for the predicate calculus is introduced and discussed.

Throughout this paper, a set of strings means a set of strings on some fixed, large, finite alphabet $\Sigma$. certain recursive set of strings on this alphabet, and we are interested in the problem of finding a good lower bound on its possible recognition times. We provide no such lower bound here, but theorem 1 will give evidence that {tautologies} is a difficult set to recognize, since many apparently difficult problems can be reduced to determining tautologyhood. By reduced we mean, roughly speaking, that if tautologyhood could be decided instantly (by an "oracle") then these problems could be decided in polynomial time. In order to make this notion precise, we introduce query machines, which are like Turing machines with oracles in [1].

A query machine is a multitape Turing machine with a distinguished tape called the query tape, and three distinguished states called the query state, yes state, and no state, respectively. If $M$ is a query machine and $T$ is a set of strings, then a T-computation of $M$ is a computation of $M$ in which

# Nathan Taylor, Liquid Type Syatems, PWL Sep 29 2023



# Constraint Satisfaction Problems

A constraint satisfaction problem consists of three components, X , D, and C:

X is a set of variables, {X1, . . . , Xn }.

D is a set of domains, {D1, . . . , Dn }, one for each variable.

C is a set of constraints that specify allowable combinations of values.

Russell, S. J., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.) Chapter 6

# Types of CSPs

- Boolean Satisfiability Problem (SAT)
- Satisfiability Modulo Theories (SMT)
- Mixed Integer Programming (MIP)
- Answer Set Programming (ASP)

NEWLY AVAILABLE SECTION OF
THE CLASSIC WORK

# The Art of Computer Programming

VOLUME 4
Satisfiability

FASCICLE 6

DONALD E. KNUTH

NEWLY AVAILABLE SECTIONS OF
THE CLASSIC WORK

# The Art of Computer Programming

VOLUME 4
Constraint Satisfaction

FASCICLE 7

DONALD E. KNUTH

*I can't get no . . .*

— MICK JAGGER and KEITH RICHARDS, *Satisfaction* (1965)

Section 7.1.1 discussed the embarrassing fact that nobody has ever been able to come up with an efficient algorithm to solve the general satisfiability problem, in the sense that the satisfiability of any given formula of size $N$ could be decided in $N^{O(1)}$ steps. Indeed, the famous unsolved question "does P = NP?" is equivalent to asking whether such an algorithm exists. We will see in Section 7.9 that satisfiability is a natural progenitor of every NP-complete problem.*

On the other hand enormous technical breakthroughs in recent years have led to amazingly good ways to approach the satisfiability problem. We now have algorithms that are much more efficient than anyone had dared to believe possible before the year 2000. These so-called "SAT solvers" are able to handle industrial-strength problems, involving millions of variables, with relative ease, and they've had a profound impact on many areas of research such as computer-aided verification. In this section we shall study the principles that underlie modern SAT-solving procedures.

# Solutions

- Exhaustive Search - intuitively exponential
- Backtracking
  - reduction/simplification rules
  - Heurisitics for selecting next variable
  - Clause learning (adding conflicts if some part is know to be not solvalble)
  - Efficient indexed data structures
  - Back jumping

Paul Butler, A gentle introduction to optimization, Rust NYC, Aug 12 2025

Video not yet available. Slides and code at https://github.com/paulgb/scip-talk

## Constraints

- No participant sends a card to themself.
- No participant sends more cards than they signed up for.
- Every participant receives as many cards as they receive.
- No pair of participants send and receive from each other.

## Objective

Maximize number of cards sent

# NFL Schedule as an optimization problem

Creating the @NFL Schedule with Gurobi https://www.youtube.com/watch?v=njISXhlc1Rw

## PARTIAL LIST OF 2019 RULES & CONSTRAINTS

Teams playing in London home or close the week prior, on BYE (or home?) the week after
Two teams must play Thursday of Week 14 after playing Thanksgiving Day
No team plays more than two road games against team coming off their BYE
No team plays consecutive road games involving cross-country trips unless requested
All teams playing road Thursday games are home the previous week
All teams playing home Thursday games have limited travel the previous week
Teams can travel no more than 2 time zones for a Thursday game
No team has earliest BYE in consecutive seasons
Manage 3-game road trips (and 3-game home stands for teams with ticket issues)
Minimize number of teams playing road game after road MNF
Minimize number of division series that end in first half of season
Minimize number of division series that are played less than 3 weeks apart
Minimize number of teams that play two consecutive road games to start or end season
Maximize number of late-season division games
Minimize early-season 1pm games for teams with weather concerns
CBS/FOX have at least three 1pm games every week, preferably geographically diverse
CBS/FOX have at least five total games, preferably six or more when doubleheader network
CBS and FOX shouldn't "lose" their key teams or an entire division all on same weekend

# russcip vs. z3

z3: Theorem Prover - SMT solver

- Microsoft
- Open Source https://github.com/Z3Prover/z3
- 2012
- Static Checking

SCIP: Solving Constraint Integer Programs

- Zeus Institute Berlin
- Open Source https://www.scipopt.org/
- 2007?
- Planning and Scheduling, Fastest non comercial solver

# Which to use?

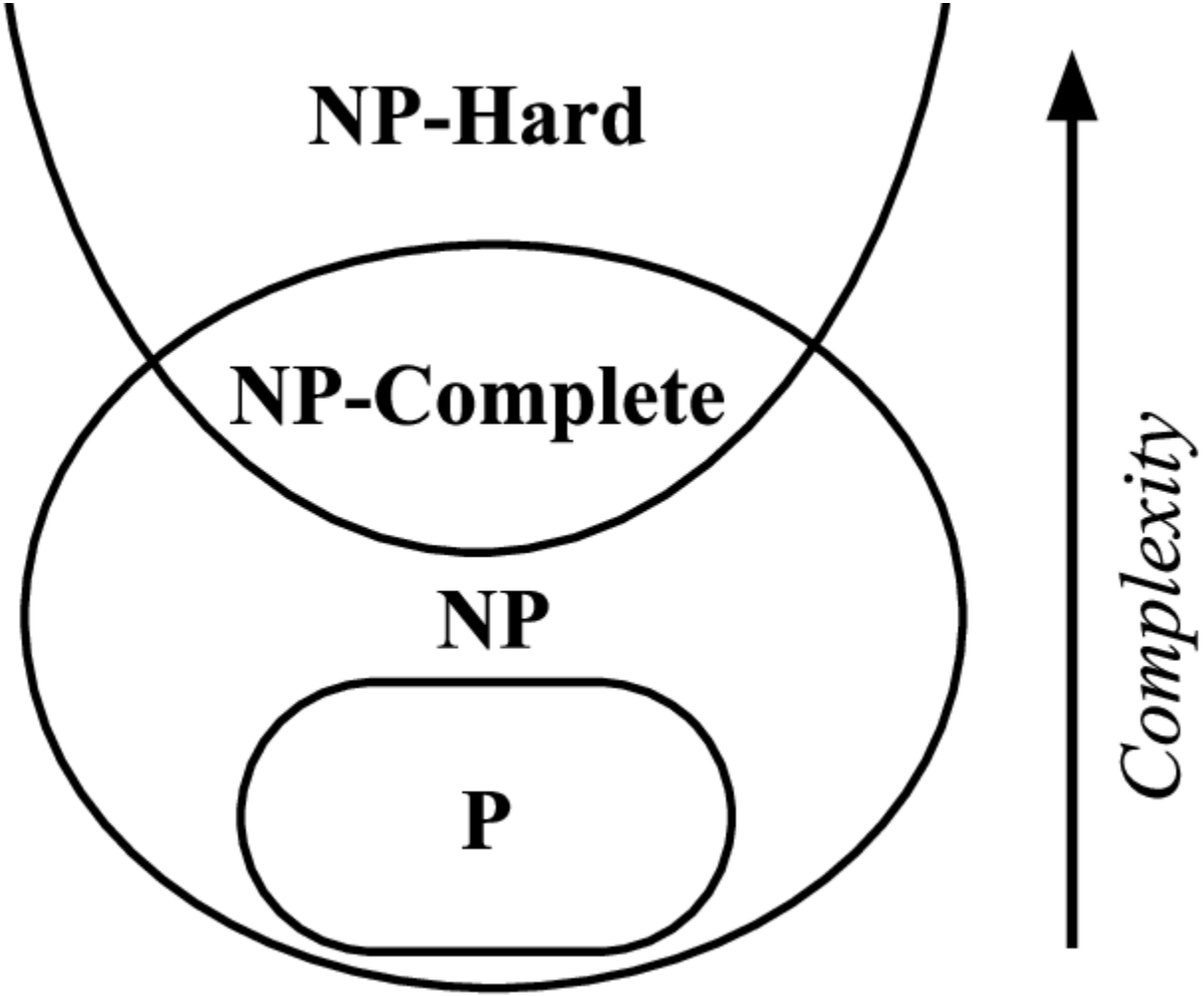
SAT solvers are fast and powerful, but don’t map to many real world problems.

Is the problem satisfaction or optimization?

- SMT solvers if the problem has few solutions, and any one will do (z3 is usually recommended here)
- If optimization is required, find the least expressive tool that totally expresses the problem
  - MiniZinc
  - Google OR-Tools
  - …
  - Tradeoff between expressability and speed

# SMT solvers are widely used

- Find differences in firewall rule sets
- Synthesize code from specifications
- Formal verification
  - Uses a “trick”, by asking the solver to prove the negation of a property. If it can’t find a solution, than there exists a value that makes the property true.
  - Dafny is a formal verification tool from Microsoft that uses this.


NP-Hard
NP-Complete
NP
P
Complexity

## Many Hard Leetcode Problems are Easy Constraint Problems

Given a set of coin denominations, find the minimum number of coins required to make change for a given number.

- For USA coinage and 37 cents, the minimum number is four (quarter, dime, 2 pennies).
- Intuitive greedy algorithm fails for non “well behaved” denominations, e.g. coin values 10, 9, and 1
- Many other examples.

Hillel Wayne, Computer Things, September 10, 2025 https://buttondown.com/hillelwayne/archive/many-hard-leetcode-problems-are-easy-constraint/

# Ways to organize a Pickleball tournament

- Teams of 2, single elimination
- Or:
  - Teams of 3
  - Grouped so that the skill levels of all teams is as close as possible
  - 1st and 2nd players from each team play first
  - Then 1st and 3rd players play
  - If required, 2nd and 3rd players from each team play

| Rating | Description | Skills Expected |
|---|---|---|
| 1.0 | Complete Beginner | No experience, basic understanding of rules |
| 2.0 | Beginner | Basic strokes, can serve occasionally |
| 2.5 | Advanced Beginner | Can sustain short rallies, basic court positioning |
| 3.0 | Developing Player | Consistent serves and returns, basic shot control |
| 3.5 | Intermediate Player | Good shot placement, understanding of strategy |
| 4.0+ | Advanced Player | High skill level, minimal unforced errors |
| 5.0+ | Elite Player | Superior skills, strategic play, very few errors |
| 6.0+ | Professional Player | Mastery of all aspects of the game |

# setup

```rust
use std::fs::File;
use std::io::{BufRead, BufReader};
use russcip::prelude::*;
use russcip::variable::*;

fn solve_grouping(ids: Vec<String>, scores: Vec<f64>) {
    let n = ids.len();
    if n % 3 != 0 {
        println!("Error: Input count ({}) must be a multiple of 3.", n);
        return;
    }

    let num_groups = n / 3;
    let total_score: f64 = scores.iter().sum();
    let target_sum = total_score / num_groups as f64;
```

# Initialize model

```
let mut model = Model::new()
    .hide_output()
    .include_default_plugins()
    .create_prob("pickle")
    .set_obj_sense(ObjSense::Minimize);
```

# Decision variables

```
let mut x = Vec::new();
for i in 0..n {
    let mut row = Vec::new();
    for j in 0..num_groups {
        let variable = model.add_var(
            0.0,
            1.0,
            0.0,
            &format!("x_{}_{}", i, j),
            VarType::Binary
        );
        row.push(variable);
    }
    x.push(row);
}
```

# Deviation variables

```
let big_m = 1e20;
let mut d = Vec::new();
for j in 0..num_groups {
    let variable = model.add_var(
        0.0,
        big_m,
        1.0,
        &format!("d_{}", j),
        VarType::Continuous
    );
    d.push(variable);
}
```

# Constraint: Each item must be in exactly one group

```
for i in 0..n {
    let vars: Vec<&Variable> = (0..num_groups).map(|j| &x[i][j]).collect();
    let coeffs = vec![1.0; num_groups];
    model.add_cons(vars, &coeffs, 1.0, 1.0, &format!("item_once_{}", i));
}
```

# Constraint: Each group must have exactly 3 items

```
for j in 0..num_groups {
    let vars: Vec<&Variable> = (0..n).map(|i| &x[i][j]).collect();
    let coeffs = vec![1.0; n];
    model.add_cons(vars, &coeffs, 3.0, 3.0, &format!("group_size_{}", j));
}
```

# Absolute Deviation Constraints (Linearizing Variance)

```
for j in 0..num_groups {
    let mut vars = (0..n).map(|i| &x[i][j]).collect::<Vec<&Variable>>();
    vars.push(&d[j]);

    // d[j] - Sum(scores * x) >= -target_sum
    let mut coeffs_up = scores.iter().map(|s| -*s).collect::<Vec<f64>>();
    coeffs_up.push(1.0);
    model.add_cons(vars.clone(), &coeffs_up, -target_sum, big_m, &format!("dev_up_{}", j));

    // d[j] + Sum(scores * x) >= target_sum
    let mut coeffs_low = scores.clone();
    coeffs_low.push(1.0);
    model.add_cons(vars, &coeffs_low, target_sum, big_m, &format!("dev_low_{}", j));
}
```

# Solve

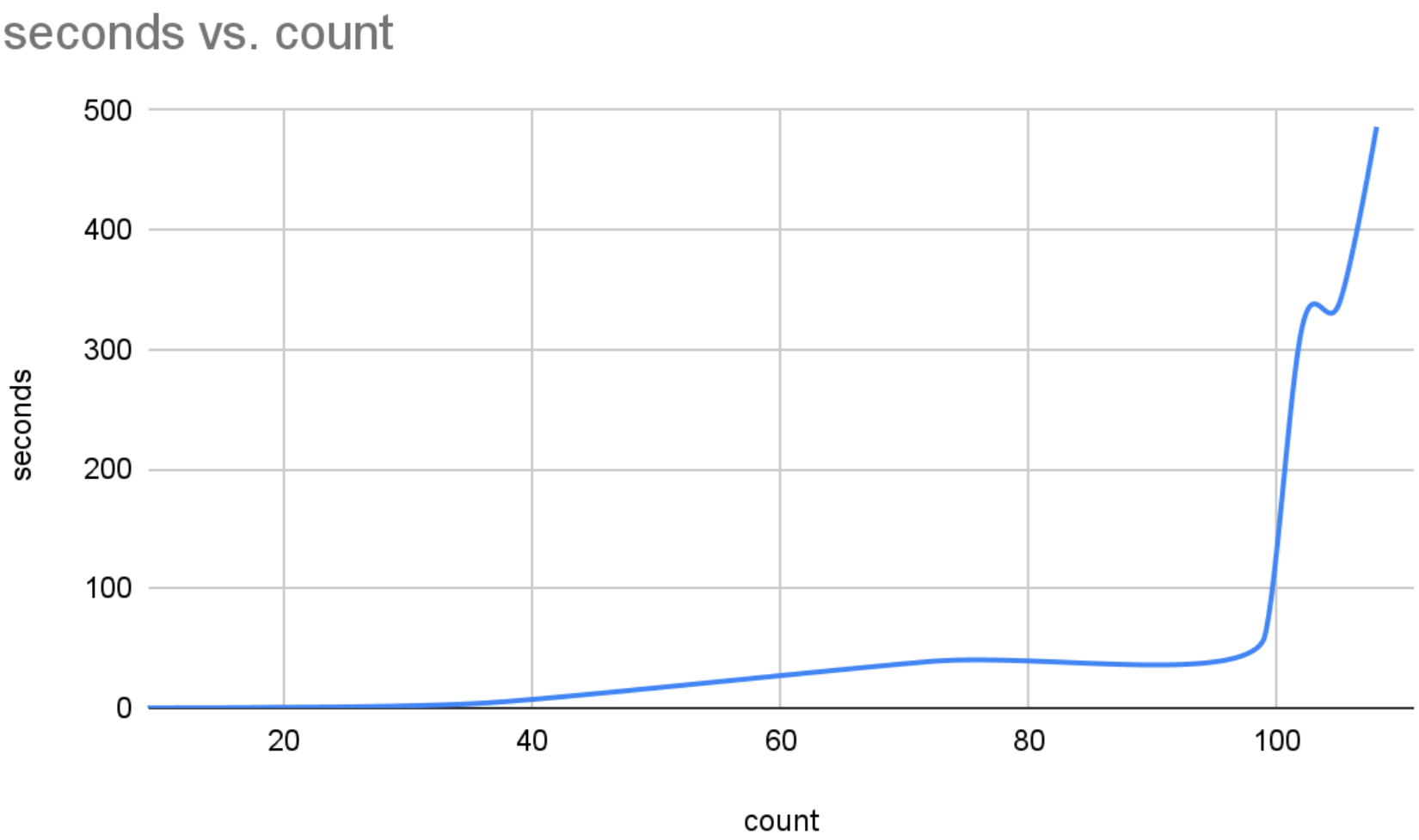
```
let solved_model = model.solve();
let status = solved_model.status();

if status == Status::Optimal {
    let sol = solved_model.best_sol().expect("No solution found");

    println!("\nTarget Average Sum: {:.2}", target_sum);
    for j in 0..num_groups {
        let mut members = Vec::new();
        let mut current_sum = 0.0;
        for i in 0..n {
            if sol.val(&x[i][j]) > 0.5 {
                members.push(&ids[i]);
                current_sum += scores[i];
            }
        }
        println!("Group {}: {:?} | Sum: {:.2} (Dev: {:.2})",
            j + 1, members, current_sum, (current_sum - target_sum).abs());
    }
} else {
    println!("Solver Status: {:?}", status);
}
```

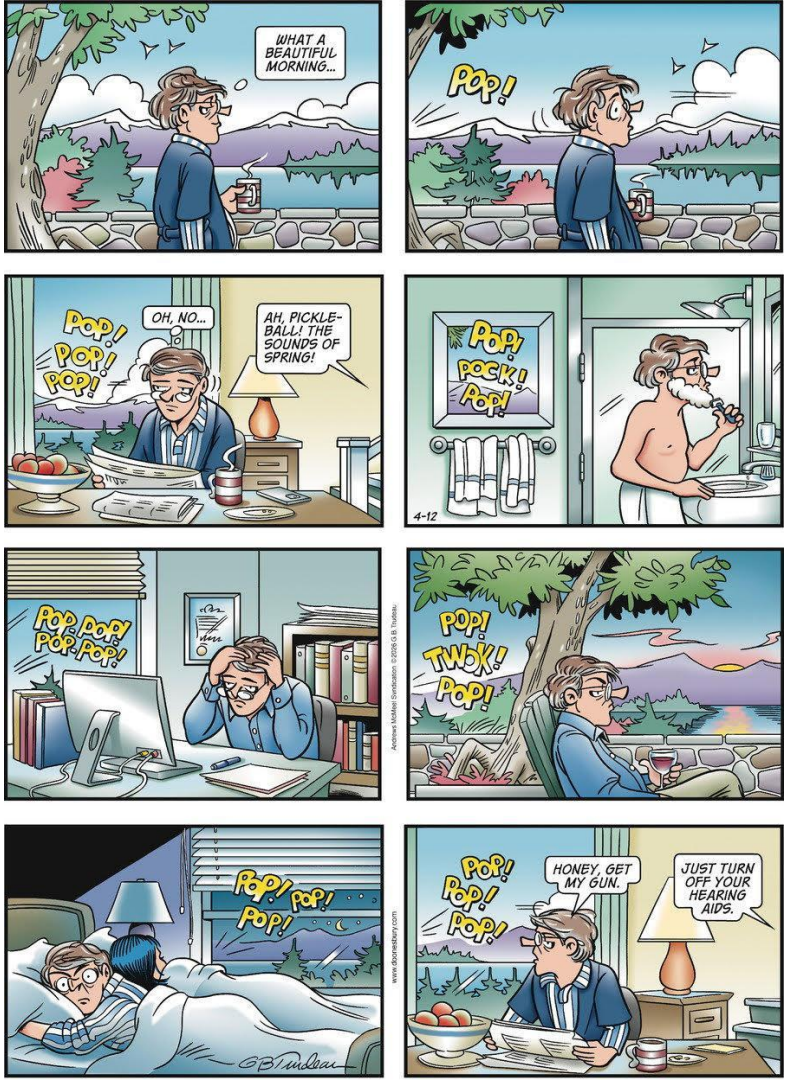
# main

```rust
fn main() {
   // let ids = vec!["A", "B", "C", "D", "E", "F"]
       // .into_iter().map(String::from).collect();
   // let scores = vec![1.0, 2.0, 3.0, 4.0, 5.0, 6.0];

   let file = File::open("data99").expect("Unable to open file");
   let reader = BufReader::new(file);
   let mut ids = Vec::new();
   let mut scores = Vec::new();
   for line in reader.lines() {
       let line = line.expect("Unable to read line");
       let parts: Vec<&str> = line.split(',').collect();
       ids.push(parts[0].to_string());
       scores.push(parts[1].parse::<f64>().expect("Invalid score"));
   }
   solve_grouping(ids, scores);
}
```

```
14:54:11 hp17 pickle$ cargo run
   Compiling scip-sys v0.1.26
   Compiling russcip v0.9.1
   Compiling pickle v0.1.0 (/home/bob/src/pickle)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 14.48s
     Running `target/debug/pickle`

Target Average Sum: 8.97
Group 1: ["AD", "CO", "CT"] | Sum: 8.50 (Dev: 0.47)
Group 2: ["AG", "BF", "BW"] | Sum: 8.50 (Dev: 0.47)
Group 3: ["AA", "AV", "DH"] | Sum: 9.00 (Dev: 0.03)
Group 4: ["AI", "AL", "BG"] | Sum: 9.00 (Dev: 0.03)
Group 5: ["AN", "CF", "CZ"] | Sum: 9.00 (Dev: 0.03)
Group 6: ["CG", "DG", "DI"] | Sum: 9.00 (Dev: 0.03)
Group 7: ["BK", "BR", "CK"] | Sum: 9.00 (Dev: 0.03)
Group 8: ["AH", "CI", "CR"] | Sum: 9.00 (Dev: 0.03)
Group 9: ["CY", "DB", "DK"] | Sum: 9.00 (Dev: 0.03)
Group 10: ["AY", "BM", "BU"] | Sum: 9.00 (Dev: 0.03)
Group 11: ["AP", "CD", "CN"] | Sum: 9.00 (Dev: 0.03)
Group 12: ["BH", "CS", "DA"] | Sum: 9.00 (Dev: 0.03)
Group 13: ["AR", "DF", "DR"] | Sum: 9.00 (Dev: 0.03)
Group 14: ["AT", "BC", "CX"] | Sum: 9.00 (Dev: 0.03)
Group 15: ["AK", "BQ", "CW"] | Sum: 9.00 (Dev: 0.03)
Group 16: ["AO", "BJ", "DO"] | Sum: 9.00 (Dev: 0.03)
Group 17: ["AZ", "CB", "CC"] | Sum: 9.00 (Dev: 0.03)
Group 18: ["AF", "CL", "DS"] | Sum: 9.00 (Dev: 0.03)
Group 19: ["AQ", "AW", "BI"] | Sum: 9.00 (Dev: 0.03)
Group 20: ["AC", "DL", "DM"] | Sum: 9.00 (Dev: 0.03)
Group 21: ["AM", "BP", "CP"] | Sum: 9.00 (Dev: 0.03)
Group 22: ["BA", "BZ", "DP"] | Sum: 9.00 (Dev: 0.03)
Group 23: ["AS", "BS", "CJ"] | Sum: 9.00 (Dev: 0.03)
Group 24: ["BB", "DT", "DU"] | Sum: 9.00 (Dev: 0.03)
Group 25: ["AJ", "BT", "CM"] | Sum: 9.00 (Dev: 0.03)
Group 26: ["BN", "BX", "CQ"] | Sum: 9.00 (Dev: 0.03)
Group 27: ["AB", "AU", "CU"] | Sum: 9.00 (Dev: 0.03)
Group 28: ["AX", "CA", "DN"] | Sum: 9.00 (Dev: 0.03)
Group 29: ["BV", "CH", "DC"] | Sum: 9.00 (Dev: 0.03)
Group 30: ["AE", "BL", "DE"] | Sum: 9.00 (Dev: 0.03)
Group 31: ["BE", "BY", "DQ"] | Sum: 9.00 (Dev: 0.03)
Group 32: ["CE", "CV", "DJ"] | Sum: 9.00 (Dev: 0.03)
Group 33: ["BD", "BO", "DD"] | Sum: 9.00 (Dev: 0.03)
```

```
15:12:02 hp17 pickle$ cat data99
AA,3.0
AB,1.0
AC,4.0
AD,2.0
AE,5.0
AF,3.0
AG,2.5
AH,4.0
AI,1.5
AJ,3.5
AK,2.0
AL,4.5
AM,5.0
AN,1.0
AO,3.0
AP,2.5
AQ,4.0
AR,1.5
AS,3.5
AT,2.0
AU,4.5
AV,5.0
AW,1.0
AX,3.0
AY,2.5
AZ,4.0
BA,1.5
BB,3.5
BC,2.0
BD,4.5
BE,5.0
BF,1.0
BG,3.0
BH,2.5
BI,4.0
BJ,1.5
BK,3.5
BL,2.0
BM,4.5
BN,5.0
BO,1.0
BP,3.0
BQ,2.5
BR,4.0
BS,1.5
BT,3.5
BU,2.0
BV,4.5
BW,5.0
BX,1.0
BY,3.0
BZ,2.5
CA,4.0
CB,1.5
CC,3.5
CD,2.0
CE,4.5
CF,5.0
CG,1.0
CH,3.0
CI,2.5
CJ,4.0
CK,1.5
CL,3.5
CM,2.0
CN,4.5
CO,5.0
CP,1.0
CQ,3.0
CR,2.5
CS,4.0
CT,1.5
CU,3.5
CV,2.0
CW,4.5
CX,5.0
CY,1.0
CZ,3.0
DA,2.5
DB,4.0
DC,1.5
DD,3.5
DE,2.0
DF,4.5
DG,5.0
DH,1.0
DI,3.0
DJ,2.5
DK,4.0
DL,1.5
DM,3.5
DN,2.0
DO,4.5
DP,5.0
DQ,1.0
DR,3.0
DS,2.5
DT,4.0
DU,1.5
15:12:06 hp17 pickle$
```


seconds vs. count
500
400
300
200
100
0
seconds
20
40
60
80
100
count




Andrews McMeel Syndication. ©2026 G.B. Trudeau.
www.doonesbury.com

# Slides and code:

https://github.com/bobmcn/pickle